Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ingeniería
Ingeniería en Ciencias y Sistemas
Compiladores 2

Ing Mario Tobar

MANUAL TÉCNICO

Diego Alessandro Gonzalez Bravo / 202230939

Quetzaltenango, 26 de septiembre
2026

Descripción general

El sistema es un compilador con interfaz gráfica que recibe un proyecto escrito en tres lenguajes:

Lenguaje	**Extensión**	**Rol dentro del proyecto**
Pig Latin	.pig	Archivo principal. Declara variables globales, funciones propias y la sección MAIOR> con el programa a ejecutar. Importa los demás archivos.
Y?	.y	Define estructuras (%estructuras) y funciones (%funciones). Los bloques se delimitan por indentación, como en Python.
Zetariano	.z	Define una clase por archivo, con atributos, constructores y métodos. El archivo debe llamarse igual que la clase.

A partir del archivo .pig el compilador:

1. Analiza léxica y sintácticamente cada archivo, siguiendo las importaciones.
2. Une todos los módulos en un solo AST y lo valida semánticamente.
3. Genera código de tres direcciones (C3D) en forma de cuartetas.
4. Traduce las cuartetas a un archivo C que se compila con gcc.

Los errores de cualquier fase se muestran en una tabla con su tipo, descripción, línea, columna y archivo.

Requisitos y tecnologías

Herramienta	**Versión**	**Uso**
Java (JDK)	21	Lenguaje de implementación. El proyecto usa expresiones switch, instanceof con patrones, record y bloques de texto.
Apache Maven	3.9 o superior	Construcción, dependencias y generación de los analizadores.
ANTLR 4	4.13.1	Generador de lexers y parsers (antlr4-maven-plugin y antlr4-runtime).
Java Swing	incluido en el JDK	Interfaz gráfica. No se usan librerías visuales externas.
GCC	cualquier versión reciente	Compilar el archivo .c generado. No lo usa el compilador en sí.

La única dependencia de ejecución es org.antlr:antlr4-runtime:4.13.1.

Compilación y ejecución del proyecto

Generar los analizadores

Las gramáticas están en src/main/antlr4/. El plugin de ANTLR genera los lexers, parsers, visitors y listeners en src/main/java/Backend/antlr/ con el paquete Backend.antlr. Se regeneran en la fase generate-sources de Maven, que también se ejecuta al compilar:

mvn generate-sources

Los archivos de Backend/antlr/ no se editan a mano: cualquier cambio se pierde al regenerar

Construir y ejecutar

mvn clean package
java -jar target/Proyecto1-compi-1.0-SNAPSHOT-jar-with-dependencies.jar

maven-assembly-plugin genera un jar ejecutable que incluye el runtime de ANTLR y tiene Main.Main como clase principal. Desde un IDE basta con ejecutar la clase Main.Main.

Compilar el código C generado

Después de guardar el código desde la pestaña **CODIGO GENERADO** (por defecto salida.c):

gcc salida.c -o salida -lm

./salida

Fase 1: análisis léxico, sintáctico y módulos

Gramáticas

Las tres gramáticas se escribieron sin código embebido (sin acciones ni predicados en Java). Toda la lógica vive en las clases Java, lo que permite regenerar los analizadores sin perder cambios.

Gramática	Regla inicial	Estructura del programa	Comentarios
pigLatin.g4	programa	imports, VARIABILES> (opcional), MUNERA> (opcional), MAIOR> ... FINIS (obligatoria)	// línea y ## bloque ##
Y.g4	programa	%estructuras (opcional) y %funciones (obligatoria)	// línea y /* bloque */
Zetariano.g4	programa	Una sola definicionClase	// línea y /* bloque */

Los tres lenguajes distinguen mayúsculas de minúsculas. En Pig Latin, finis cierra bloques (si, dum, per, funciones) y FINIS cierra la sección MAIOR>.

Tipos primitivos de cada lenguaje y su categoría interna (Type.TypeCategory):

Categoría	Pig Latin	Y?	Zetariano
INT	numerus	entero	int
FLOAT	decimalis	flotante	double
CHAR	littera	caracter	char
BOOLEAN	bool (verum, falsus)	bool (verdadero, falso)	boolean (true, false)
STRING	textum	cadena	String
STRUCT	estructura importada de un .y	estructura	no aplica
CLASS	clase importada con novus	no aplica	class
ARRAY	series nombre[n] : tipo	tipo nombre[n]	tipo[], new tipo[n]

Además existen VOID (funciones sin retorno), NULL (null de Zetariano) y ERROR (tipo interno que reciben las expresiones inválidas para que el análisis continúe).

Instrucciones de entrada y salida:

Operación	Pig Latin	Y?	Zetariano
Imprimir con salto de línea	>> a >> b;	imprimir(a)	println(a)
Imprimir sin salto	no aplica	no aplica	print(a)
Leer	variable <<	leer()	readln()

En Pig Latin cada valor impreso lleva su propio >>. Si una línea nueva empieza con >>, PigLatinAstVisitor la trata como una impresión separada.

Indentación en Y? (YIndentTokenSource)

Y? delimita los bloques por indentación. ANTLR no genera esos tokens por sí mismo, así que la gramática declara tokens { INDENT, DEDENT } y la clase YIndentTokenSource se coloca entre el lexer y el parser:

YLexer -> YIndentTokenSource -> CommonTokenStream -> YParser

Funcionamiento:

- El lexer produce NEWLINE con los espacios de la línea siguiente incluidos. Una tabulación cuenta como 4 espacios (ESPACIOS_POR_TABULACION).
- Una Pila<Integer> guarda los niveles de indentación abiertos; inicia con 0.
- Si la nueva línea tiene más indentación que la cima, se apila el nivel y se emite INDENT.
- Si tiene menos, se desapilan niveles y se emite un DEDENT por cada uno.
- Las líneas en blanco consecutivas se unen en un solo salto.
- Al llegar al final del archivo se cierran todos los bloques abiertos y se agrega un salto final si faltaba.

- Los tokens pendientes se guardan en una Cola<Token> y se entregan uno por uno al parser.

La gramática también acepta llaves { } como alternativa a la indentación.

Construcción del AST (AstBuilder y visitors)

AstBuilder.analizarArchivo(Path) detecta el lenguaje por la extensión y ejecuta:

1. El lexer del lenguaje (más YIndentTokenSource en Y?).
2. El parser, comenzando en la regla programa.
3. Un ListenerErroresAntlr conectado al lexer y al parser, en lugar de los listeners de consola de ANTLR.
4. El visitor correspondiente, que recorre el árbol de ANTLR y devuelve un ProgramNode:

    - PigLatinAstVisitor (extiende pigLatinBaseVisitor<Node>)
    - YAstVisitor (extiende YBaseVisitor<Node>)
    - ZetarianoAstVisitor (extiende ZetarianoBaseVisitor<Node>)

Detalles relevantes:

- Literales resuelve las secuencias de escape (\n, \t, \", \', \\, etc.) de cadenas y caracteres para los tres lenguajes.
- Cada declaración (StructDecl, ClassDecl, FunctionDecl) queda marcada con su archivo de origen (archivoOrigen). Así, un error semántico dentro de un módulo importado se reporta con el archivo correcto.
- El resultado de cada archivo es un ResultadoAnalisis con el AST, sus errores, el lenguaje y el nombre del archivo.

Coloreado de sintaxis

El coloreado usa los mismos lexers de ANTLR que el compilador, sin librerías externas:

- LenguajeColoreado (enum PIG_LATIN, Y, ZETARIANO) elige el lexer según la extensión del archivo y clasifica cada tipo de token en una CategoriaToken.
- CategoriaToken (enum) define el color, la negrita y la cursiva de cada categoría: palabra reservada, tipo de dato, sección, función del sistema, función, identificador, cadena, número, constante, operador, delimitador y comentario.

- ColoreadorSintaxis.analizar recorre los tokens y produce una lista de Segmento (inicio, fin, categoria):
    - Un identificador seguido de (se colorea como función.
    - Un identificador después de novus, new, class o estructura se colorea como tipo.
    - Las gramáticas descartan los comentarios (-> skip), así que el texto entre tokens que no es espacio en blanco se colorea como comentario.
    - Los caracteres que el lexer no reconoce se dejan sin color: el coloreado no marca errores.
    - PosicionesTexto convierte posiciones de ANTLR (puntos de código) a posiciones de Java (caracteres), para textos con emojis u otros caracteres fuera del plano básico.
- ColoreadorSintaxis.aplicar pinta los segmentos en el StyledDocument del editor.
- PestaniaEditor recolorea 200 ms después de la última tecla (un Timer de Swing). Los cambios de estilo no marcan el archivo como modificado.
- Después de compilar, Inicio.colorearSegunCompilacion reúne las clases y estructuras de todos los módulos y las pasa a cada editor (aplicarTiposDelProyecto). Así, un tipo definido en otro archivo también se colorea como tipo de dato.

12. Manejo de errores

Clase	Función
TipoError	Enum LEXICO, SINTACTICO, SEMANTICO.
ErrorCompilacion	Tipo, mensaje, línea, columna, archivo y lexema del error.
GestorErrores	Acumula los errores. Guarda el archivo actual para asignarlo a cada error, y permite unir los errores de varios archivos (agregarTodos).
ListenerErroresAntlr	Reemplaza los listeners de consola de ANTLR. Si el error trae un token (error del parser), se registra como **sintáctico**; si no (el lexer no reconoció un carácter), como **léxico**.

Los mensajes léxicos y sintácticos son los que produce ANTLR (por ejemplo missing 'finis' at 'FINIS'). Los semánticos los redacta el analizador (por ejemplo ID NO DECLARADO: x).

El parser de ANTLR se recupera de los errores y sigue analizando, de modo que en una sola compilación se reportan varios errores sintácticos. El análisis semántico tampoco se detiene en el primer error: la expresión inválida recibe el tipo ERROR y el recorrido continúa con el resto del programa.

En tiempo de ejecución, el programa C solo valida la división y el módulo entre cero.

Estructuras de datos propias

El paquete Backend.estructuras implementa estructuras genéricas enlazadas, todas iterables:

Clase	**Operaciones**	**Uso en el compilador**
Pila<T>	apilar, desapilar, cima, cimaONulo, estaVacia,tamaño	Niveles de indentación (YIndentTokenSource), ámbitos abiertos (TablaSimbolos), marcos activos (GeneradorC3D) y etiquetas de break/continue (ControladorC3D).
Cola<T>	encolar, desencolar, frente, frenteONulo, estaVacia, tamanio	Tokens INDENT/DEDENT pendientes (YIndentTokenSource).
ListaEnlazada<T>	agregar, obtener, establecer, contiene, tamanio	Disponible como estructura auxiliar.

1. Encapsulamiento en Zetariano: modificadores public, private, protected y acceso sin modificador.
2. Herencia y polimorfismo en Zetariano: extends, super, this, sobrescritura con @Override obligatorio y despacho dinámico.
3. Generación de RISC-V a partir de las cuartetas del C3D, con un ensamblador y un simulador propios que ejecutan el programa dentro de la aplicación, sin herramientas externas.

Clases nuevas y modificadas

Archivo	Tipo de cambio	Responsabilidad
src/main/antlr4/Zetariano.g4	Modificado	Modificadores de acceso, extends, super, anotaciones
Backend/Ast/ModificadorAcceso.java	Nuevo	Enum con los cuatro niveles de acceso
Backend/Ast/declaration/ClassDecl.java	Modificado	Guarda la(s) clase(s) indicadas en extends
Backend/visitor/ZetarianoAstVisitor.java	Modificado	Pasa modificadores, superclase y anotación al AST
Backend/proyecto/CompiladorProyecto.java	Modificado	Localiza automáticamente el archivo .z de la clase padre
Backend/simbolos/Simbolo.java	Modificado	Modificador, clase dueña, superclase,

		id de clase, override
Backend/semantica/AnalizadorSemantico.java	Modificado	Resolución de herencia, sobrescritura y control de acceso
Backend/semantica/TablaCompatibilidad.java	Modificado	Asignación de clase hija a clase padre
Backend/c3d/GeneradorC3D.java	Modificado	Objetos con id de clase, cadena de constructores, despacho dinámico
Backend/riscv/TraductorRiscV.java	Nuevo	Cuartetas → ensamblador RISC-V (sintaxis RARS)
Backend/riscv/EnsambladorRiscV.java	Nuevo	Ensamblador de dos pasadas
Backend/riscv/InstruccionRiscV.java	Nuevo	Instrucción ya decodificada
Backend/riscv/ProgramaRiscV.java	Nuevo	Programa ensamblado y mapa de memoria
Backend/riscv/SimuladorRiscV.java	Nuevo	Ejecución de las instrucciones y llamadas al sistema

Backend/riscv/ConsolaRiscV.java	Nuevo	Interfaz de entrada/salida del programa simulado
Backend/riscv/ErrorRiscV.java	Nuevo	Error de ensamblado o de ejecución, con número de línea
Frontend/Inicio.java	Modificado	Fases 5 y 6, consola de entrada y botón DETENER
Frontend/ModelosTablas.java	Modificado	Columna ACCESO, extends y @Override en la tabla de símbolos

Encapsulamiento

Cambios en la gramática

Se agregaron los tokens KW_PUBLIC, KW_PRIVATE y KW_PROTECTED, y la regla modificadorAcceso. El modificador es opcional en atributos, constructores y métodos:

modificadorAcceso : KW_PUBLIC | KW_PRIVATE | KW_PROTECTED ;declaracionCampo : modificadorAcceso? tipo ID (ASSIGN (expresion | literalArreglo))? SEMI ;

definicionConstructor: modificadorAcceso? ID LPAREN listaParametros? RPAREN bloque ;

definicionMetodo : ANOTACION? modificadorAcceso? tipoRetorno ID LPAREN listaParametros? RPAREN bloque ;

definicionClase : KW_PUBLIC? KW_CLASS ID (KW_EXTENDS ID (COMMA ID)*)? LBRACE miembroClase* RBRACE ;

Una clase solo puede llevar public o ningún modificador.

Representación en el AST

El enum ModificadorAcceso representa los cuatro niveles. Cada uno tiene un nivel numérico, que se usa para verificar que una sobrescritura no reduzca la visibilidad (sección 2.6):

Constante	Palabra	Nivel
PRIVADO	private	0
DEFECTO	(sin modificador)	1
PROTEGIDO	protected	2
PUBLICO	public	3

ZetarianoAstVisitor convierte el modificador de cada miembro en su constante, y asigna DEFECTO cuando no hay ninguno. El valor queda guardado en los nodos VarDeclStmt (atributo), ConstructorDecl y FunctionDecl (método).

Información en la tabla de símbolos

Al registrar cada miembro de una clase, su Simbolo guarda dos datos para el control de acceso:

- modificador: el nivel de acceso declarado.
- claseDuenia: la clase que declaró el miembro. Un atributo heredado conserva la clase dueña original. Por ejemplo, nombre dentro de Perro sigue teniendo claseDuenia = Animal.

Durante la revisión de los cuerpos, el analizador mantiene la variable claseActual: la clase cuyo código se está revisando. Vale null fuera de una clase, es decir, en el main de PigLatin, en las funciones globales y en el código Y.

La tabla de símbolos de la interfaz muestra la columna ACCESO para cada miembro.

Reglas de acceso

Por indicación del auxiliar, el proyecto no tiene paquetes. Por eso el acceso sin modificador se comporta como en Java dentro de un mismo paquete: visible desde cualquier parte.

Modificador	El acceso se permite si…
private	claseActual es la clase dueña del miembro
protected	claseActual es la clase dueña o una de sus descendientes (hija, nieta, …)
public	Siempre
(sin modificador)	Siempre

Implementación en AnalizadorSemantico.esAccesible:

```
return switch (modificador) {
                case PRIVADO -> claseActual != null && claseDuenia.equals(claseActual.getIdentificador());
   case PROTEGIDO  -> claseActual != null && claseActual.esSubclaseDe(claseDuenia);
   case PUBLICO, DEFECTO -> true;
};
```

Simbolo.esSubclaseDe(ancestro) sube por la cadena de superClase hasta encontrar el ancestro o llegar a una clase sin padre. Una clase cuenta como subclase de sí misma.

Un miembro private que se hereda sí existe en el objeto hijo y ocupa su casilla (sección 2.4); solo no puede nombrarse desde la clase hija. El encapsulamiento restringe dónde se puede escribir un nombre, no qué hay en memoria.

Puntos de verificación

validarAcceso aplica la regla anterior y reporta el error. Se llama desde cada construcción del lenguaje que puede nombrar un miembro:

Construcción	Ejemplo	Método del analizador
Atributo por nombre simple dentro de un método	energia = 0; dentro de Perro	visitIdentifierExpr
Acceso con punto	a.nombre en el main	visitMemberAccessExpr
Llamada a método (después de resolver la sobrecarga)	a.metodoPrivado()	visitCallExpr
Creación de objeto	novus Clase() con constructor private	visitNewObjectExpr
Llamada al constructor padre	super(...) hacia un constructor private	analizarLlamadaConstructorPadre

Los atributos heredados private también se insertan en el ámbito de la clase hija. Así, si la hija los nombra, el error que se reporta es de acceso, no “ID NO DECLARADO”.

Errores semánticos de encapsulamiento

Formato del mensaje: NO SE PUEDE ACCEDER AL <ATRIBUTO|METODO|CONSTRUCTOR>: <nombre> DE LA CLASE <dueña> DESDE <LA CLASE X | FUERA DE UNA CLASE>: <regla>

Caso	Mensaje generado
Perro usa energia (private de Animal)	NO SE PUEDE ACCEDER AL ATRIBUTO: energia DE LA CLASE Animal DESDE LA CLASE Perro: ES PRIVATE, SOLO ES ACCESIBLE DENTRO DE LA CLASE Animal
main.pig usa a.nombre (protected)	NO SE PUEDE ACCEDER AL ATRIBUTO: nombre DE LA CLASE Animal DESDE FUERA DE UNA CLASE: ES PROTECTED, SOLO ES ACCESIBLE DENTRO DE LA CLASE Animal Y DE SUS CLASES HIJAS

Impacto en el código generado

El encapsulamiento es una verificación exclusivamente de compilación. Ni el C3D, ni el C, ni el RISC-V contienen información de modificadores: si el programa superó el análisis semántico, ya está garantizado que ningún acceso viola las reglas. Por eso el encapsulamiento no tiene costo en tiempo de ejecución.

La única excepción es indirecta: un método private no puede sobrescribirse, así que sus llamadas nunca usan despacho dinámico (sección 2.9.6).

Herencia y polimorfismo

Cambios en la gramática

Construcción	Regla o token	Observación
class Hija extends Padre	KW_EXTENDS ID (COMMA ID)*	La gramática acepta una lista para que extends A, B produzca un error semántico claro (“HERENCIA MULTIPLE NO PERMITIDA”) en lugar de un error sintáctico
super(...) y super.metodo()	KW_SUPER → # ExprSuper	
@Override	ANOTACION : '@' [a-zA-Z_][a-zA-Z_ 0-9]* ;	Acepta cualquier anotación para que @Overide (mal escrita) llegue al análisis y se reporte “ANOTACION DESCONOCIDA”

En el AST, ClassDecl guarda la lista superClases. getSuperClase() devuelve la primera clase de la lista, y tieneSuperClase() indica si la clase hereda. FunctionDecl guarda la anotación del método.

Localización del archivo de la clase padre

Zetariano no tiene import. Cuando el visitor encuentra extends Animal, registra Animal como clase referenciada. Después, CompiladorProyecto busca el archivo Animal.z así:

1. En la carpeta del archivo que hace la referencia.
2. Si no está, en las carpetas superiores, de la más cercana a la más lejana.
3. Nunca por encima de la raíz del proyecto, que es la carpeta abierta en el explorador de la interfaz.

Cada archivo encontrado se analiza sintácticamente y se une al mismo AST. También se valida que el nombre del archivo coincida con el nombre de la clase. Si el archivo no existe, el analizador semántico reporta que la clase padre no está definida.

Registro de clases en el análisis semántico

Antes de revisar cualquier cuerpo de método, AnalizadorSemantico registra todas las clases del programa en cinco pasos. Esto permite usar una clase declarada más adelante o en otro archivo.

1. Registrar nombres cada clase recibe un símbolo CLASE y un idClase (1, 2, 3, ...)
2. resolverHerencia enlazar cada clase con su clase padre y detectar ciclos
3. Ordenar por profundidad de herencia: los padres antes que las hijas
4. registrarMiembros atributos (con su casilla), métodos y constructores
5. validarSobrescrituras reglas de @Override

Paso 1: nombres e idClase. Cada clase recibe un número único y consecutivo. Ese número se guarda en la casilla 0 de cada objeto, y en tiempo de ejecución indica la clase real del objeto (sección 2.9.6).

Paso 2: resolverHerencia. Para cada clase con extends:

Situación	Acción
Más de una clase en extends	Error de herencia múltiple; se toma la primera como padre para seguir el análisis
La clase hereda de sí misma	Error
La clase padre no existe	Error
El nombre no corresponde a una clase	Error
Caso válido	hija.setSuperClase(simboloPadre)

Después se recorre la cadena de padres de cada clase. Si se vuelve a la clase inicial (por ejemplo, A → B → A), se reporta herencia cíclica y se corta el enlace. Si no se cortara, los recorridos posteriores por la cadena de padres no terminarían.

Paso 3: orden por profundidad. profundidadHerencia cuenta los ancestros de cada clase: Animal tiene 0 y Perro tiene 1. Las clases se ordenan de menor a mayor profundidad. Así, cuando se registran los miembros de una hija, su padre ya está completo, sin importar el orden de los archivos.
Paso 4: registrarMiembrosDeClase. Ver la sección 2.4.
Paso 5: validarSobrescrituras. Ver la sección 2.6.

Distribución del objeto en memoria

Un objeto es un bloque contiguo de celdas en el heap. La distribución se decide en registrarMiembrosDeClase con estas reglas:

1. La casilla 0 contiene el idClase de la clase real del objeto.
2. Primero se copian los atributos heredados, en las mismas casillas que tienen en la clase padre.
3. Después van los atributos propios, en casilla = 1 + cantidad de atributos ya registrados.

Objeto Perro en el heap:

```
dirección del objeto ──► [0]  2        idClase de Perro
                         [1]  energia   heredado de Animal (private)
                         [2]  nombre    heredado de Animal (protected)
                         [3]  edad      heredado de Animal (public)
                         [4]  raza      propio de Perro    (default)
```

Tabla de símbolos generada para el ejemplo:

```
CLASE Animal  idClase=1  superClase=-
  atributo  energia  int     casilla=1  acceso=private    duenia=Animal
  atributo  nombre   String  casilla=2  acceso=protected  duenia=Animal
  atributo  edad     int     casilla=3  acceso=public     duenia=Animal
  ctor      Animal(String)          etiqueta=constructor_Animal_String
  metodo    sonido()                override=false  etiqueta=metodo_Animal_sonido_0
  metodo    hablar()                override=false  etiqueta=metodo_Animal_hablar_0
CLASE Perro  idClase=2  superClase=Animal
  atributo  energia  int     casilla=1  acceso=private    duenia=Animal
  atributo  nombre   String  casilla=2  acceso=protected  duenia=Animal
  atributo  edad     int     casilla=3  acceso=public     duenia=Animal
  atributo  raza     String  casilla=4  acceso=default    duenia=Perro
  ctor      Perro(String, String)   etiqueta=constructor_Perro_String_String
  metodo    sonido()                override=true   etiqueta=metodo_Perro_sonido_0
  metodo    presentar()             override=false  etiqueta=metodo_Perro_presentar_0
```

Por qué esta distribución hace posible el polimorfismo: un atributo heredado está en la misma casilla en la clase padre y en la hija. El código compilado de Animal.hablar() lee nombre en THIS + 2, y esa instrucción es correcta tanto si THIS apunta a un Animal como si apunta a un Perro. Ningún método heredado necesita compilarse de nuevo para la clase hija.

Cada método y constructor recibe una etiqueta única formada por la clase, el nombre y los tipos de sus parámetros. Si no tiene parámetros, el sufijo es _0:

Elemento	Formato de la etiqueta	Ejemplo
Método	metodo_<Clase>_<nombre>_<tipos>	metodo_Perro_sonido_0
Constructor	constructor_<Clase>_<tipos>	constructor_Perro_String_String
Valores iniciales de los atributos	init_<Clase>	init_Animal

Por eso Animal.sonido y Perro.sonido existen como funciones distintas en el C3D. Si una clase hija vuelve a declarar un atributo que ya heredó, se reporta un error.

Ámbito de la clase durante la revisión de los cuerpos

visitClassDecl prepara la revisión del código de cada clase:

1. Fija claseActual con la clase que se revisa.
2. Abre el ámbito Clase_<Nombre>, con el primer desplazamiento en 1 + atributos heredados.
3. Inserta en ese ámbito copias de los atributos heredados, con sus casillas originales (Ambito.insertarConDireccion). Así, un método de Perro puede usar nombre sin escribir this.nombre.
4. Inserta los atributos propios y revisa cada constructor y cada método.

La información de los atributos queda en dos estructuras con propósitos distintos:

Estructura	Uso
Simbolo.getMiembros() de la clase	Distribución del objeto: tamaño, casilla de cada atributo y acceso objeto.atributo
Ámbito Clase_<Nombre>	Resolver nombres simples dentro de los métodos de la clase

Sobrescritura y @Override

La firma de un método es su nombre más los tipos de sus parámetros, por ejemplo sonido() o mover(int, int). Para cada método de una clase hija, Simbolo.buscarImplementacion(firma) busca un método con la misma firma en el padre, el abuelo, y así hasta el final de la cadena.

Según lo indicado por el auxiliar, @Override es obligatorio. Ponerlo de más también es un error.

Situación	Resultado
Un ancestro tiene la firma y el método lleva @Override	Sobrescritura válida; esOverride = true
Un ancestro tiene la firma y el método no lleva @Override	Error: falta la anotación
Lleva @Override y ningún ancestro tiene la firma	Error: anotación de más
Lleva @Override y la clase no hereda de nadie	Error
El método del ancestro es private	No se puede sobrescribir: con @Override es error; sin la anotación es un método nuevo e independiente
Cambia el tipo de retorno	Error; solo se acepta devolver una clase hija del tipo original (retorno covariante)
Reduce la visibilidad (nivel menor, sección 1.2)	Error
Anotación distinta de @Override	Error: anotación desconocida

super y this

- super(argumentos) llama al constructor de la clase padre sobre el mismo objeto. Solo se permite como primera instrucción de un constructor. visitConstructorDecl activa la bandera superPermitido únicamente para la instrucción 0, y solo si es una llamada a super. Un super(...) en otra posición encuentra la bandera apagada y produce un error. El constructor del padre se elige por sobrecarga (según los tipos de los argumentos) y se verifica que sea accesible.
- super.metodo(...): super tiene el tipo de la clase padre, así que el método se busca a partir del padre. Llama directamente a la versión del padre, sin despacho dinámico.
- this tiene el tipo de la clase actual. Igual que super, solo puede usarse dentro de un método o constructor.